

SELEKSI LABORATORIUM INTELIGENSI BUATAN

Task #2

Spesifikasi Tugas Besar - Local Search

Write-up - DTL, Logistic Regression, dan SVM

Ariel Cornelius Sitorus 13524085

Daftar Isi

1. Spesifikasi Local Search

1. Tujuan dan Deskripsi Permasalahan
2. Representasi State
3. Aturan dan Batasan (Constraints)
4. Formulasi Initial State (Random)
5. Successor / Neighbor Function
6. Objective / Heuristic Function
7. Implementasi Wajib
8. Daftar Fungsi / Command Utama
9. Contoh Input dan Output
10. Referensi

2. Write-up DTL, Logistic Regression, dan SVM

- TLDR
- Problem Overview
- DTL (Decision Tree Learning)
- LR (Logistic Regression)
- SVM
- Validation Strategy
- Percobaan yang Gagal
- Pengembangan Lebih Lanjut
- Referensi

Spesifikasi Tugas Besar: Local Search - Data Center Network Topology Design

1. Tujuan dan Deskripsi Permasalahan

Sebuah data center memiliki **N node** (server rack / network switch) yang harus saling terhubung melalui kabel jaringan sehingga membentuk sebuah **topologi** (graf tak berarah). Tujuan perancangan adalah menemukan topologi yang:

1. **Meminimalkan latensi rata-rata** antar node (semakin sedikit hop antar node, semakin baik),
2. **Meminimalkan biaya pemasangan kabel** (kabel makin panjang secara fisik, makin mahal),
3. **Memaksimalkan fault tolerance / redundansi**, yaitu meminimalkan jumlah kabel yang menjadi *single point of failure* (SPOF),

dengan batasan jumlah **port** (derajat maksimum) yang dimiliki tiap node terbatas.

Karena kualitas sebuah topologi hanya bergantung pada *konfigurasi akhirnya secara utuh* - bukan pada urutan pemasangan kabel atau jalur menuju suatu "goal" tertentu - persoalan ini secara natural dapat diformulasikan sebagai **local search problem**: setiap kandidat topologi adalah satu *complete state*, dan pencarian dilakukan dengan bergerak antar state melalui operasi lokal (menambah / menghapus / memindah satu kabel), bukan dengan membangun solusi secara bertahap dari kosong menuju goal.

Problem ini dipilih karena punya trade-off yang secara nyata membuatnya non-trivial: menambah kabel menurunkan latensi & menaikkan fault tolerance, tetapi menaikkan biaya - sehingga strategi naif seperti "pasang semua kabel yang mungkin" atau "hapus semua kabel" sama-sama buruk, dan local search benar-benar dibutuhkan untuk mencari titik keseimbangan.

2. Representasi State

Diberikan **N node** tetap dengan posisi fisik 2 dimensi $p_i = (x_i, y_i)$, $i = 0..N-1$ (posisi rak di

lantai data center). Posisi ini dibangkitkan **sekali** di awal sebagai bagian dari instance masalah (bukan bagian dari variabel yang dioptimasi - yang dioptimasi hanyalah *edge* mana saja yang terpasang).

State s adalah sebuah graf tak berarah dengan $G = (V, E)$ tetap, dan $V = \{0, \dots, N-1\}$ $E \subseteq \{\{i, j\} : i \neq j\}$ adalah himpunan kabel yang saat ini terpasang.

Representasi konkret: matriks adjacency biner simetris A berukuran $N \times N$:

$$\begin{aligned} A[i][j] &= A[j][i] = 1 && \text{jika kabel } \{i, j\} \text{ terpasang} \\ A[i][j] &= A[j][i] = 0 && \text{jika tidak} \\ A[i][i] &= 0 && \text{(tidak ada self-loop)} \end{aligned}$$

Ini adalah **complete-state formulation**: satu matriks A merepresentasikan rancangan topologi secara **lengkap** - tidak ada informasi "riwayat pembangunan" atau "path" yang disimpan/dibutuhkan.

Daftar Variabel Instance

Entitas	Atribut	Contoh	Keterangan
Node	<code>id</code>	<code>0</code>	Index integer <code>0..N-1</code> , merepresentasikan satu server rack / network switch.
Node	<code>posisi (x, y)</code>	<code>(42.3, 71.8)</code>	Koordinat 2D pada grid lantai data center (satuan meter). Dibangkitkan sekali secara acak per instance dan tetap - bukan bagian dari state yang dioptimasi.
Node	<code>K_max</code>	<code>4</code>	Kapasitas port (derajat maksimum) node tersebut. Dalam PoC dipakai nilai seragam untuk semua node, tapi struktur data mendukung nilai per-node berbeda.
Edge (kabel)	<code>(i, j)</code>	<code>(2, 7)</code>	Pasangan node yang dihubungkan satu kabel fisik.
Edge (kabel)	<code>status</code>	<code>1 / 0</code>	Terpasang atau tidak - disimpan sebagai entri <code>A[i][j]</code> pada adjacency matrix.
State	<code>A</code>	matriks <code>12x12</code> boolean, simetris	Representasi lengkap seluruh kabel yang terpasang pada satu kandidat topologi (lihat Bagian 2).
Bobot objective	<code>w_L, w_C, w_B, w_D, PENALTI_BESAR</code>	<code>1.0, 1.0, 0.5, 1.0, 50</code>	Konstanta pembobot tiap komponen objective function (lihat Bagian 6), bagian dari instance masalah - bukan dioptimasi.

3. Aturan dan Batasan (Constraints)

- **Degree cap (hard constraint):** setiap node `i` punya jumlah port terbatas `K_max`. $\deg(i) = \sum_j A[i][j] \leq K_max$ untuk semua `i`. Constraint ini dijaga langsung di successor function - tidak pernah dibangkitkan state yang melanggarnya.
- **Tidak ada self-loop:** `A[i][i] = 0`.
- **Tidak ada multi-edge:** dijamin otomatis karena representasi biner (satu kabel per pasang node).
- **Konektivitas (soft constraint):** graf **tidak wajib** selalu *connected* di setiap state antara (misalnya state hasil satu langkah mutasi GA), namun **dipenalti sangat berat** pada objective function karena topologi yang terputus berarti sebagian node data center tidak bisa saling berkomunikasi. Konektivitas didekati sebagai soft constraint (lewat penalti besar) alih-alih hard constraint, supaya successor function tetap sederhana dan proses pencarian tetap bisa "melewati" state yang sementara terputus jika itu membuka jalan ke solusi lebih baik.

4. Formulasi Initial State (Random)

Prosedur `random_initial_state(N, K_max, positions, seed)`:

1. Inisialisasi A = matriks nol $N \times N$.
2. Bangun sebuah **random spanning tree** di atas permutasi acak node - menjamin graf awal **connected** sejak awal, sehingga objective function initial state langsung memberi sinyal yang informatif (bukan selalu bernilai penalti-disconnect yang sama besarnya). Tiap edge ditambahkan hanya jika kedua endpoint belum melebihi K_{max} .
3. Tambahkan beberapa **edge tambahan secara acak** (selama tidak melanggar degree cap) untuk memberi redundansi awal pada topologi, sejumlah percobaan acak proporsional terhadap N .
4. Kembalikan A sebagai initial state.

Bagian yang bersifat acak adalah **pemilihan pasangan node mana yang dihubungkan**; validitas terhadap constraint (degree cap) selalu dijaga.

5. Successor / Neighbor Function

Tiga jenis **move** lokal yang mengubah state A menjadi A' dengan mengubah tepat satu bagian kecil dari topologi:

Move	Syarat	Efek
<code>ADD_EDGE(i, j)</code>	$A[i][j] = 0$, $\deg(i) < K_{max}$, $\deg(j) < K_{max}$	pasang kabel baru $\{i,j\}$
<code>REMOVE_EDGE(i, j)</code>	$A[i][j] = 1$	lepas kabel $\{i,j\}$
<code>REWIRE(i, j, k)</code>	$A[i][j] = 1$, $A[i][k] = 0$, $\deg(k) < K_{max}$, $k \neq j$	pindahkan salah satu ujung kabel dari node j ke node k (derajat i tidak berubah)

`REWIRE` penting sebagai move tersendiri (bukan sekadar remove+add dua langkah terpisah) karena pada kondisi banyak node sudah mencapai K_{max} , kombinasi add/remove terpisah bisa membuat pencarian macet - `REWIRE` memungkinkan reorganisasi topologi tanpa pernah melanggar degree cap di langkah antara.

- `get_all_neighbors(A)` - mengembalikan **seluruh** state tetangga valid yang dapat dicapai dengan tepat satu move di atas. Dipakai oleh **Hill-Climbing steepest-ascent**, yang mengevaluasi semua neighbor lalu berpindah ke yang terbaik.
- `get_random_neighbor(A)` - mengembalikan **satu** neighbor acak (memilih jenis move secara acak, lalu parameter move secara acak). Dipakai oleh **Simulated Annealing** karena jumlah total neighbor bisa berorde $O(N^2)$, sehingga enumerasi penuh setiap iterasi tidak efisien.

6. Objective / Heuristic Function

Karena permasalahan bersifat multi-kriteria (latensi, biaya, fault tolerance), ketiganya digabung menjadi **satu nilai cost** yang **diminimalkan**:

$$\text{cost}(A) = w_L \cdot L(A) + w_C \cdot C(A) + w_B \cdot B(A) + w_D \cdot D(A) \cdot \text{PENALTI_BESAR}$$

dengan:

- ♦ **L(A)** - **proxy latensi**: rata-rata *shortest-path* (jumlah hop, BFS unweighted) antar seluruh pasang node yang berada dalam komponen sama. Untuk pasangan node yang **tidak** terhubung (beda komponen), dipakai jarak sentinel $2N$ (bukan infinity) agar tetap menghasilkan angka *finite* yang tetap memberi sinyal gradien untuk mendekatkan mereka.
- ♦ **C(A)** - **biaya kabel**: total $\sum \text{euclidean_distance}(p_i, p_j)$ atas seluruh edge $\{i, j\} \in E$,
dinormalisasi terhadap biaya *complete graph* (sum jarak seluruh pasangan node) agar skalanya sebanding dengan komponen objective lainnya.
- ♦ **B(A)** - **proxy fault tolerance**: jumlah **bridge edge**, yaitu edge yang jika dihapus akan memecah graf menjadi lebih banyak komponen (dihitung dengan algoritma pencarian bridge berbasis DFS/Tarjan, $O(V+E)$). Sebuah bridge adalah *single point of failure*: satu-satunya jalur yang menyokong konektivitas di sekitarnya. Semakin sedikit bridge, semakin banyak jalur redundant, semakin fault-tolerant topologinya.
- ♦ **D(A)** - **penalti disconnect**: $(\text{jumlah connected component}) - 1$. Bernilai 0 jika graf connected (kondisi ideal), dan dikalikan **PENALTI_BESAR** karena node yang terisolasi dari jaringan data center adalah kegagalan desain fatal.

Alasan pemilihan: objective ini dipilih (bukan sekadar "minimalkan jumlah kabel" atau "maksimalkan konektivitas" saja) karena secara eksplisit merepresentasikan **trade-off nyata** dalam desain jaringan: setiap term saling menarik ke arah berlawanan, sehingga solusi optimal adalah hasil keseimbangan, bukan solusi ekstrem yang trivial untuk ditemukan.

Bobot default yang dipakai pada PoC: $w_L = 1.0$, $w_C = 1.0$, $w_B = 0.5$, $w_D = 1.0$,
PENALTI_BESAR = 50.

Contoh Perhitungan Objective Function (Worked Example)

Misalkan instance kecil dengan $N = 5$ node, $K_{\max} = 3$, posisi:

Node	Posisi (x, y)
0	(0, 0)
1	(10, 0)
2	(10, 10)
3	(0, 10)
4	(5, 5)

dan state dengan edge terpasang (siklus 0-1-2-3-0

A $E = \{(0, 1), (1, 2), (2, 3), (3, 0), (0, 4)\}$

ditambah satu kabel pendant ke node 4). Degree tiap node: $\text{deg}(0)=3$, $\text{deg}(1)=2$, $\text{deg}(2)=2$,
 $\text{deg}(3)=2$, $\text{deg}(4)=1$ - semua $\leq K_{\max}=3$, valid.

1. **L(A)** (**latensi**) - BFS hop antar 10 pasang node: $(0, 1)=1$, $(0, 2)=2$, $(0, 3)=1$, $(0, 4)=1$,
 $(1, 2)=1$, $(1, 3)=2$, $(1, 4)=2$, $(2, 3)=1$, $(2, 4)=3$, $(3, 4)=2$. Total = 16, dibagi 10 pasang $\rightarrow L(A)$
= 1.6.

2. **C(A) (biaya kabel)** - jarak edge terpasang: $d(0,1)=10$, $d(1,2)=10$, $d(2,3)=10$, $d(3,0)=10$, $d(0,4)=\sqrt{50}\approx 7.07$, total ≈ 47.07 . Jarak seluruh 10 pasang node (complete graph) totalnya ≈ 96.57 . $C(A) = 47.07 / 96.57 \approx 0.4875$.
3. **B(A) (bridge)** - edge (0,4) adalah satu-satunya bridge (menghapusnya mengisolasi node 4); keempat edge pada siklus 0-1-2-3-0 bukan bridge karena punya jalur alternatif. $B(A) = 1$.
4. **D(A) (disconnect)** - graf connected (1 komponen), jadi $D(A) = 1 - 1 = 0$.

Maka:

$$\begin{aligned}
 \text{cost}(A) &= w_L \cdot L(A) + w_C \cdot C(A) + w_B \cdot B(A) + w_D \cdot D(A) \cdot \text{PENALTI_BESAR} \\
 &= 1.0(1.6) + 1.0(0.4875) + 0.5(1) + 1.0(0)(50) \\
 &= 1.6 + 0.4875 + 0.5 + 0 \\
 &= 2.5875
 \end{aligned}$$

Jika kemudian $\text{REWIRE}(0, 4, 2)$ diterapkan (memindahkan ujung kabel dari node 4 ke node 2, selama $\text{deg}(2) < K_{\text{max}}$), edge (0,4) yang tadinya bridge SPOF hilang, tapi node 4 justru terisolasi total ($D(A)$ naik ke $0.5 \dots \cdot 50$ bernilai besar) - ilustrasi konkret mengapa neighbor sembarangan bisa

memperburuk cost meski mengurangi satu term, dan mengapa pencarian perlu mengevaluasi cost **gabungan**, bukan term individual.

7. Implementasi Wajib

- Representasi state & successor/neighbor function → Bagian 2 & 5.
- Objective/heuristic function → Bagian 6.

7.1 Hill-Climbing - keempat varian diimplementasikan (bonus)

Varian	Cara kerja	Alasan keberadaannya
Steepest-Ascent (Basic)	Evaluasi seluruh neighbor via <code>get_all_neighbors</code> , pindah ke yang cost-nya terendah; berhenti bila tak ada yang lebih baik.	Baseline greedy - deterministik, selalu berakhir di local optimum basin awalnya.
Sideways Move	Sama seperti steepest-ascent, tetapi neighbor ber-cost sama juga diterima, dibatasi <code>max_sideways=25</code> langkah mendatar berturut-turut.	Steepest-ascent berhenti begitu tidak ada perbaikan ketat, padahal di <i>plateau</i> jalan buntu itu bisa jadi semu. Batas langkah mencegah berputar-putar selamanya di dataran datar.
Stochastic	Kumpulkan seluruh neighbor yang memperbaiki cost, lalu pilih satu secara acak seragam .	Steepest-ascent bersifat deterministik: dari state yang sama selalu menempuh jalur sama dan jatuh ke local optimum yang sama. Pemilihan acak membuat lintasan bervariasi antar run sehingga berpeluang mendarat di basin lain.
Random Restart	Jalankan steepest-ascent dari <code>n_restarts=10</code> initial state acak yang independen, ambil hasil terbaik.	Satu kali hill-climbing hanya bisa mencapai optimum dari basin tempatnya berangkat. Restart menyampel basin-basin berbeda, sehingga hasil terbaiknya mendekati optimum global seiring bertambahnya restart.

7.2 Simulated Annealing dan Genetic Algorithm

- **Simulated Annealing:** pada tiap iterasi, ambil satu `get_random_neighbor`. Jika cost-nya lebih baik, selalu pindah. Jika lebih buruk, tetap pindah dengan probabilitas `exp(-Δcost / T)`. Temperature menurun tiap iterasi mengikuti *geometric cooling* $T \leftarrow T \times \text{cooling_rate}$.
- **Genetic Algorithm:** representasi kromosom = bit-vector dari upper-triangle matriks adjacency (panjang `N(N-1)/2`, tiap bit menyatakan ada/tidaknya satu edge kandidat).
 - **Selection:** tournament selection (ambil `k` individu acak, pilih yang cost-nya terendah).
 - **Crossover:** uniform crossover dua parent menghasilkan dua child, diikuti langkah **repair** (`repair_degree`) yang menghapus edge secara acak pada node yang melanggar `K_max` - dibutuhkan karena crossover dua parent valid tidak menjamin child valid.
 - **Mutation:** flip bit acak (toggle edge) dengan probabilitas `mutation_rate`, dilanjutkan `repair_degree` bila melanggar constraint.
 - Populasi berevolusi selama `generations` generasi; individu terbaik yang pernah ditemukan (*best-so-far*) selalu disimpan.

8. Daftar Fungsi / Command Utama

Fungsi	Deskripsi
<code>generate_positions(N, seed)</code>	Bangkitkan posisi 2D acak untuk N node
<code>random_initial_state(N, K_max, seed)</code>	Bangun initial topologi random valid (spanning tree + edge tambahan)
<code>get_degree(A, i)</code>	Derajat (jumlah kabel) node <code>i</code>
<code>get_all_neighbors(A, K_max)</code>	Enumerasi seluruh state tetangga valid (add/remove/rewire)
<code>get_random_neighbor(A, K_max, rng)</code>	Satu neighbor acak
<code>avg_shortest_path(A)</code>	Rata-rata jarak BFS antar semua pasang node (proxy latensi)
<code>count_bridges(A)</code>	Jumlah bridge edge via DFS/Tarjan
<code>count_components(A)</code>	Jumlah connected component via BFS
<code>cable_cost(A, positions)</code>	Total biaya kabel euclidean, dinormalisasi
<code>objective(A, positions, weights)</code>	Nilai cost gabungan - dipakai konsisten oleh ketiga algoritma
<code>hill_climbing_steepest_ascent(A0, K_max, positions, max_iter)</code>	HC varian Basic - pindah ke neighbor terbaik
<code>hill_climbing_sideways(A0, K_max, positions, max_iter, max_sideways)</code>	HC varian Sideways Move - menerima neighbor ber-cost sama, dibatasi budget
<code>hill_climbing_stochastic(A0, K_max, positions, max_iter, rng)</code>	HC varian Stochastic - pilih acak di antara neighbor yang memperbaiki
<code>hill_climbing_random_restart(N, K_max, positions, n_restarts, rng)</code>	HC varian Random Restart - ulangi dari beberapa initial state acak, kembalikan (<code>best</code> , <code>history</code> , <code>restart_costs</code>)
<code>simulated_annealing(A0, K_max, positions, T0, cooling_rate, max_iter)</code>	Local search SA
<code>ga_crossover(A1, A2)</code>	Uniform crossover representasi bit-vector + repair
<code>ga_mutate(A, K_max, mutation_rate)</code>	Mutasi bit-flip + repair
<code>repair_degree(A, K_max)</code>	Perbaiki pelanggaran degree cap dengan menghapus edge acak
<code>genetic_algorithm(N, K_max, positions, pop_size, generations, ...)</code>	Local search Genetic Algorithm
<code>describe_state(A, positions)</code>	Cetak edge list, derajat tiap node, dan breakdown nilai objective

9. Contoh Input dan Output

Representasi Instance (literal)

Instance masalah (posisi node, `K_max`, bobot) direpresentasikan sebagai berikut, mengambil contoh 5-node yang dipakai pada worked example Bagian 6:

```
instance = {
  "n": 5,
  "k_max": 3,
  "positions": [
    {"id": 0, "x": 0, "y": 0},
    {"id": 1, "x": 10, "y": 0},
    {"id": 2, "x": 10, "y": 10},
    {"id": 3, "x": 0, "y": 10},
    {"id": 4, "x": 5, "y": 5},
  ],
  "weights": {"w_L": 1.0, "w_C": 1.0, "w_B": 0.5, "w_D": 1.0},
  "penalty_disconnect": 50.0,
}
```

State (topologi) direpresentasikan sebagai adjacency matrix `A`, contoh untuk edge set `{(0, 1), (1, 2), (2, 3), (3, 0), (0, 4)}` pada Bagian 6:

```

  0  1  2  3  4
0 [0, 1, 0, 1, 1]
1 [1, 0, 1, 0, 0]
2 [0, 1, 0, 1, 0]
3 [1, 0, 1, 0, 0]
4 [1, 0, 0, 0, 0]
```

yang setara dengan edge list `[(0, 1), (0, 3), (0, 4), (1, 2), (2, 3)]`, `degree = [3, 2, 2, 2, 1]`, dan `cost(A) = 2.5875` (lihat perhitungan lengkap di Bagian 6).

Hasil Eksekusi PoC (`N = 12`)

Input: `N = 12` node, `K_max = 4` port/node, posisi acak pada grid `100 × 100` meter, bobot default (lihat Bagian 6).

Output (hasil aktual eksekusi `src/local_search/main.py`, `seed=42`; log lengkap dan grafik konvergensi ada di `notebooks/local_search/experiment.ipynb` serta `src/local_search/pic/objective_convergence.png`):

```

=== INITIAL STATE (random spanning tree + extra edges) ===
Jumlah edge : 18
Degree/node : [4, 4, 2, 4, 4, 1, 3, 2, 2, 4, 4, 2]
cost=2.7453 | latency=2.0000 | cable_cost=0.2453 | bridges=1 | components=1

=== HILL-CLIMBING 1/4 (Steepest-Ascent) ===
Berhenti di local optimum setelah 6 iterasi.
Final: cost=1.9625 | latency=1.6667 | cable_cost=0.2959 | bridges=0 | components=1

=== HILL-CLIMBING 2/4 (Sideways Move) ===
max_sideways=25. Final: cost=1.9625 (identik dengan Steepest-Ascent)

=== HILL-CLIMBING 3/4 (Stochastic) ===
Final: cost=1.9524 | latency=1.6667 | bridges=0 | components=1

=== HILL-CLIMBING 4/4 (Random Restart) ===
10 restart. Cost akhir tiap restart:
1.9805, 1.9565, 1.9697, 1.9329, 1.9792, 2.0120, 1.9143, 1.9825, 1.9707, 1.9334
Terbaik: cost=1.9143 | latency=1.6515 | cable_cost=0.2628 | bridges=0 | components=1

=== SIMULATED ANNEALING ===
3000 iterasi, T0=10.0, cooling_rate=0.997.
Best: cost=1.9290 | latency=1.6515 | cable_cost=0.2775 | bridges=0 | components=1

=== GENETIC ALGORITHM ===
pop_size=40, 150 generasi, crossover_rate=0.8, mutation_rate=0.03.
Best: cost=1.9481 | latency=1.6364 | cable_cost=0.3117 | bridges=0 | components=1

=== RINGKASAN ===

```

Algoritma	Cost Awal	Cost Akhir
HC - Steepest-Ascent	2.7453	1.9625
HC - Sideways Move	2.7453	1.9625
HC - Stochastic	2.7453	1.9524
HC - Random Restart	(acak)	1.9143
Simulated Annealing	2.7453	1.9290
Genetic Algorithm	(acak)	1.9481

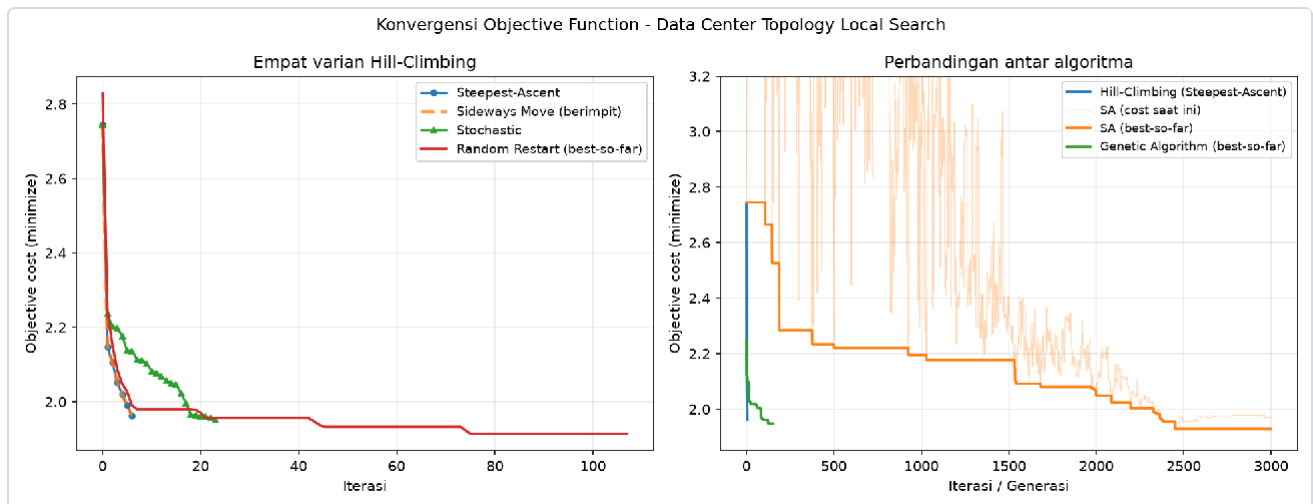
Seluruh algoritma konsisten menurunkan cost dari topologi awal yang punya 1 bridge (single point of failure) menjadi topologi akhir dengan **0 bridge** (sepenuhnya fault-tolerant) dan tetap *connected* - menunjukkan search berhasil menyeimbangkan trade-off latency/biaya/redundansi sesuai desain objective function pada Bagian 6.

Analisis perbandingan varian. Urutan hasilnya sendiri informatif:

1. **Random Restart terbaik (1.9143)**, bahkan mengungguli Simulated Annealing. Sebaran cost antar restart (1.9143 s.d. 2.0120, rentang ± 0.10) memperlihatkan bahwa ruang pencarian ini memang penuh local optimum yang kualitasnya berbeda-beda. Justru karena itu, menyampel banyak titik awal lebih efektif daripada menyempurnakan satu lintasan.

2. **Sideways Move memberi hasil identik dengan Steepest-Ascent (1.9625)** - dan ini bukan bug. Objective function pada Bagian 6 bernilai *riil kontinu* (didominasi suku `cable_cost` euclidean), sehingga dua neighbor bernilai **persis sama** hampir tidak pernah terjadi; plateau yang menjadi alasan keberadaan varian ini praktis tidak muncul, jadi budget sideways-nya tidak pernah terpakai. Varian ini akan relevan pada objective berbasis pencacahan integer (mis. jumlah bridge saja), yang plateau-nya banyak.
3. **Stochastic (1.9524) sedikit lebih baik dari Steepest-Ascent** meski keduanya berangkat dari state yang sama - memilih acak di antara neighbor yang memperbaiki membuatnya menempuh jalur berbeda dan berakhir di local optimum yang lain.
4. **Simulated Annealing (1.9290) unggul atas seluruh HC single-start** berkat penerimaan probabilistik yang memungkinkannya lolos dari local optimum yang menjebak Steepest-Ascent hanya setelah 6 iterasi.

Bonus - visualisasi konvergensi (`src/local_search/main.py` → `pic/objective_convergence.png`): panel kiri membandingkan keempat varian Hill-Climbing, panel kanan membandingkan antar keluarga algoritma (HC vs SA vs GA) pada instance yang sama.



10. Referensi

- [1] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. [Accessed: Aug. 8, 2026].
- [2] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Aug. 8, 2026].
- [3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: <https://www.youtube.com/watch?v=L0b0CN3Fds8>. [Accessed: Aug. 8, 2026].
- [4] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. [Accessed: Aug. 8, 2026].
- [5] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. [Accessed: Aug. 8, 2026].
- [6] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2021, ch. 4.1 ("Local Search and Optimization Problems") - rujukan untuk keempat varian Hill-Climbing, Simulated Annealing, dan Genetic Algorithm.
- [7] R. Tarjan, "A note on finding the bridges of a graph," *Information Processing Letters*, vol. 2, no. 6, pp. 160-161, 1974 - algoritma pencarian bridge $O(V+E)$ yang dipakai pada komponen $B(A)$ objective function (Bagian 6).

Write-up: DTL, Logistic Regression, dan SVM - Loan Approval Classification

TLDR

Kompetisi Kaggle memprediksi `loan_status` (disetujui/ditolak) dari data demografis dan finansial pemohon pinjaman, dievaluasi dengan macro F1-score. Tiga algoritma - Decision Tree Learning (CART), Logistic Regression, dan SVM - diimplementasikan from scratch (numpy only) dan dibandingkan dengan implementasi scikit-learn yang setara; ketiganya berhimpit dengan pembandingnya (selisih ≤ 0.002), memvalidasi kebenaran implementasi. **DTL (CART) adalah model terbaik: macro F1 CV 0.8747.**

Dua perbaikan yang terbukti secara empiris: **cost-complexity pruning** menggantikan pre-pruning pada CART (0.8701 $\rightarrow$ 0.8747, sekaligus menyederhanakan tree dari 60 menjadi 34 leaf), dan **log1p** pada kolom uang yang sangat skewed (LR & SVM masing-masing +0.005). Sama pentingnya, empat pendekatan lain diuji dan **gagal** - class weighting, threshold tuning, feature engineering turunan, dan SVM kernel RBF - masing-masing dengan angka dan penjelasan penyebabnya.

Temuan analisis yang paling menentukan: ketimpangan kelas 3,5:1 pada dataset ini **semu**. Ia berasal dari satu cabang deterministik (50,8% data selalu ditolak), sementara subproblem yang benar-benar dipelajari model justru nyaris seimbang 55:45 - fakta yang menjelaskan mengapa class weighting merugikan. Analisis plafon menunjukkan **0.873-0.875 adalah batas nyata di dalam aturan** (enam pendekatan gagal menembusnya), sementara sinyal yang tersisa hanya terjangkau lewat ensemble yang dilarang.

Problem Overview

Dataset **Loan Approval Classification** berisi 28.800 baris data latih (22.400 ditolak / 6.400 disetujui - imbalanced $\pm 3.5:1$) dan 7.200 baris data uji, dengan 12 fitur demografis, finansial, dan riwayat kredit. Tidak ada missing value pada kedua file.

Evaluasi kompetisi menggunakan **macro F1-score** (rata-rata F1 kelas 0 dan kelas 1 tanpa mempertimbangkan proporsi), yang dipilih justru karena distribusi kelas timpang - model yang hanya menebak mayoritas (kelas 0) akan mendapat macro F1 rendah meski akurasi tinggi.

Temuan EDA penting:

- `previous_loan_defaults_on_file` = "Yes" berkorelasi **sempurna** dengan `loan_status` = 0 (100% baris dengan riwayat gagal bayar pasti ditolak di data latih) - fitur ini sangat prediktif dan menjadi split alami pertama bagi decision tree.
- Ditemukan outlier data entry: `person_age` hingga 144 tahun (4 baris) dan `person_emp_exp` hingga 125 tahun (7 baris) - di-clip ke batas masuk akal (100 dan 60) sebelum training.
- `person_home_ownership`, `loan_int_rate`, dan `loan_percent_income` juga menunjukkan asosiasi yang jelas terhadap `loan_status`.

Konsekuensi penting dari temuan pertama: dataset ini sebenarnya terbelah dua.

<code>previous_loan_defaults_on_file</code>	label 0	label 1
Yes	14.629	0
No	7.771	6.400

Cabang **Yes** (50,8% data) **deterministik** dan diselesaikan satu split; sisa 14.171 baris yang benar-benar harus dipelajari justru **nyaris seimbang (55:45)**. Artinya label "imbalanced 3,5:1" itu sepenuhnya artefak cabang deterministik, dan **seluruh error model hidup di 14.171 baris tersebut** - akurasi model final di sana hanya 0.83, sementara baris sisanya otomatis benar. Temuan ini yang menjelaskan kegagalan class weighting (lihat Percobaan yang Gagal).

Preprocessing (lihat `src/dtl_lr_svm/data.py`):

- Clip outlier `person_age` (≤ 100) dan `person_emp_exp` (≤ 60).
- log1p** pada `person_income` dan `loan_amnt`. `person_income` sangat *right-skewed* (skewness ≈ 37 - ada income jutaan bersebelahan dengan median ± 60 ribu). Setelah z-score, segelintir baris ekstrem itu mendominasi skala dan mengencet mayoritas data ke pita sempit, yang merugikan model berbasis gradien. **log1p** mengembalikan sebaran yang bisa dipakai: **LR +0.0046 dan SVM +0.0046**. CART kebal terhadap transformasi monoton apa pun, dan memang terverifikasi skoranya identik (0.8703 sebelum & sesudah) - sehingga transformasi ini aman diterapkan global dan pipeline tetap satu untuk ketiga model.
- Binary-encode `person_gender` dan `previous_loan_defaults_on_file`.
- One-hot encode `person_home_ownership` (4 kategori).
- Standardisasi (z-score) fitur numerik - dibutuhkan Logistic Regression & SVM (gradient-based), tidak dibutuhkan CART (split berbasis threshold tidak sensitif terhadap skala).
- `person_id` dibuang dari fitur (hanya identifier, tidak prediktif).

Class-balanced sample weighting ($w_c = n / (n_classes \cdot n_c)$) juga diimplementasikan dan tersedia di ketiga model (parameter `sample_weight`), dengan hipotesis awal akan membantu macro F1 pada data imbalanced ini - namun secara empiris justru **menurunkan** performa (lihat Percobaan yang Gagal), sehingga tidak dipakai pada konfigurasi final.

DTL (Decision Tree Learning)

Varian: CART (Gini impurity, binary split). Dipilih karena dataset memiliki campuran fitur numerik (income, credit_score, dst.) dan kategorikal (setelah encoding menjadi biner) - CART menangani split biner untuk keduanya secara native tanpa perlu diskritisasi manual seperti ID3, dan lebih ringan diimplementasikan dari C4.5 (tidak perlu gain ratio + penanganan continuous-attribute yang lebih rumit). CART juga membuatnya apple-to-apple dengan `DecisionTreeClassifier` scikit-learn yang juga berbasis Gini.

Implementasi (`src/dtl_lr_svm/algoritma/decision_tree.py`): pencarian threshold terbaik per fitur divektorisasi dengan numpy - data diurutkan per fitur, lalu **cumulative weighted class count** dipakai untuk mengevaluasi *seluruh* kandidat threshold sekaligus dalam satu operasi numpy (bukan loop per-threshold), sehingga tetap efisien untuk 28.800 baris. Sample weight diakumulasi sebagai *weighted* Gini, bukan raw count.

Bonus - cost-complexity pruning (optimasi di luar materi kelas). Ukuran tree awalnya diatur dengan *pre-pruning* (`max_depth=8`, `min_samples_leaf=20`). Grid penuh atas `max_depth` × `min_samples_leaf` (30 kombinasi)

ternyata **tidak pernah melewati 0.8703**, dan setelah awal sudah persis berada di puncak grid itu - pre-pruning sudah mentok. Masalah mendasarnya: pre-pruning harus memutuskan berhenti **sebelum** tahu apakah subtree di bawahnya akan berguna.

Minimal cost-complexity pruning (Breiman et al. [4], [5]) membalik urutannya: tumbuhkan tree besar dulu (`max_depth=16`, `min_samples_leaf=5`), baru pangkas berdasarkan apa yang **benar-benar dicapai** tiap subtree. Tree dinilai dengan $R_{\alpha}(T) = R(T) + \alpha \times (\text{jumlah leaf})$, dengan $R(T)$ = total Gini cost berbobot

pada leaf. Tiap leaf harus "membayar sewa" α , jadi subtree hanya layak dipertahankan bila akurasi yang dibelinya menutup sewa leaf tambahannya. Subtree optimal dicari lewat satu *dynamic programming* bottom-up: di tiap node bandingkan $R(t) + \alpha$ (jadikan leaf) dengan $\text{best_cost(kiri)} + \text{best_cost(kanan)}$ (pertahankan split),

ambil yang lebih murah - karena nilai tiap anak sudah optimal untuk subtree-nya, satu lintasan post-order menyelesaikan seluruh tree.

Hasilnya menembus plafon pre-pruning: **0.8701** → **0.8747**, dengan tree akhir hanya **34 leaf** (vs 60) - lebih akurat *sekaligus* lebih sederhana. $\alpha = 3 \cdot 10^{-4}$ dipilih lewat sweep CV; optimumnya landai ($2 \cdot 10^{-4}$ - $3 \cdot 10^{-4}$ selisih ≤ 0.0002) sehingga tidak rapuh. Perlu dicatat jujur bahwa α dipilih pada CV yang sama yang dilaporkan, jadi 0.8747 sedikit optimistis. Sebagai validasi kebenaran, versi from-scratch diadu dengan `ccp_alpha` scikit-learn pada beberapa α dan menghasilkan **jumlah leaf identik** (38 vs 38, 25 vs 25, 19 vs 19) serta skor sama.

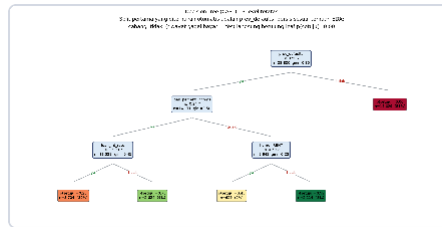
Agar keunggulan ini tidak sekadar keberuntungan satu pembagian data, CV diulang dengan **8 seed berbeda**: pruning menang **8 dari 8 seed** (rata-rata +0.0034, rentang +0.0008 s.d. +0.0048). Rata-rata lintas seed adalah **0.8736 ± 0.0019** untuk pruning vs 0.8702 ± 0.0013 untuk pre-pruning - jadi angka 0.8747 pada tabel (seed 42) sedikit di sisi beruntung, dan ±0.874 adalah ekspektasi yang lebih jujur.

Perbandingan langsung keduanya menegaskan perbedaan sifatnya. Tanpa pruning, memperbesar tree merugikan: `depth=24`, `min_samples_leaf=1` jatuh ke **0.8463**. (Menariknya `depth=24`, `min_samples_leaf=50` tetap 0.8703 - kedua parameter itu saling menggantikan, yang lebih ketat duluan yang menentukan ukuran, sehingga `max_depth` besar tidak berbahaya selama `min_samples_leaf` sudah menghentikan pertumbuhan.) Dengan pruning, memperbesar tree justru aman lalu **jenuh**: grow depth 8/12/16/20/24 → 0.8728 / 0.8742 / 0.8747 / 0.8747 / 0.8747. Inilah keunggulan pokoknya

- kompleksitas ekstra tidak lagi berubah menjadi overfitting karena bagian yang tidak terbukti berguna dibuang belakangan. Setelah `depth=16` yang dipakai sudah berada tepat di titik jenuh itu, jadi menaikannya lagi tidak memberi apa pun.

Bug yang sempat ditemukan & diperbaiki: (a) `right_w0` salah rumus ($\frac{\text{total_w0} - \text{right_w1}}{\text{total_w0} - \text{left_w0}}$) membuat pencarian split menemukan split "sempurna" palsu, tree berhenti sebagai satu leaf, macro F1 anjlok ke ~0.18; (b) `min_samples_leaf` semula hanya dicek *setelah* threshold terbaik dipilih - bila kandidat terbaik ternyata ilegal, node langsung dijadikan leaf padahal ada split legal yang sedikit lebih buruk. Sekarang constraint itu diberlakukan **di dalam** pencarian threshold.

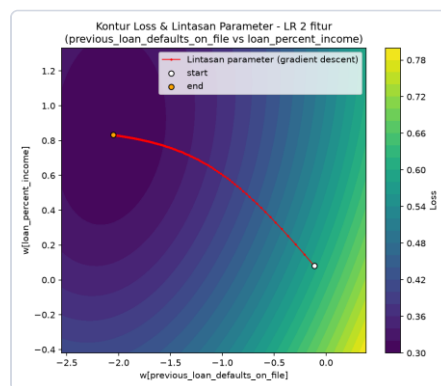
Bonus - visualisasi percabangan tree (`src/dtl_lr_svm/bonus_visualizations.py` → `pic/tree_plot.png`); hanya 3 level teratas yang digambar demi keterbacaan - model final untuk submission adalah tree hasil pruning dengan 34 leaf. Warna leaf menyatakan `p(disetujui)`: merah = ditolak, hijau = disetujui). Root split yang ditemukan otomatis oleh algoritma adalah `previous_loan_defaults_on_file`, tepat sesuai temuan EDA - cabang "riwayat gagal bayar = Yes" langsung berujung leaf dengan `proba(disetujui)=0.00`.



LR (Logistic Regression)

Implementasi (`src/dtl_lr_svm/algoritma/logistic_regression.py`): sigmoid + binary cross-entropy loss dengan regularisasi L2, dioptimasi via **full-batch gradient descent** (vektorisasi numpy penuh, `lr=0.5`, `n_epochs=800`, `l2=1e-3`). Loss & lintasan parameter direkam tiap epoch untuk keperluan diagnostik/bonus.

Bonus - visualisasi training (`bonus_visualizations.py`): `pic/lr_loss_curve.png` menunjukkan loss (weighted BCE + L2) menurun mulus dan konvergen dalam ± 150 epoch. `pic/lr_loss_contour.png` melatih model bantu 2-fitur (`previous_loan_defaults_on_file` vs `loan_percent_income` , dipilih karena keduanya paling berasosiasi dengan target di EDA) supaya ruang parameter berdimensi 2 dan bisa digambar konturnya secara langsung - lintasan gradient descent terlihat bergerak mulus menuruni kontur elliptical menuju titik loss minimum (loss turun monoton dari ± 0.58 ke ± 0.28 dalam 800 epoch pada model 14-fitur, lihat `pic/lr_loss_curve.png` di repo).



SVM

Implementasi (`src/dtl_lr_svm/algoritma/svm.py`): SVM linear dioptimasi dengan algoritma **Pegasos** (Shalev-

Shwartz et al., 2007) - mini-batch stochastic sub-gradient descent pada objective primal $(\lambda/2) ||w||^2 + \text{hinge-loss}$, dengan learning rate $\eta_t = 1/(\lambda t)$. Pegasos dipilih dibanding menyelesaikan dual QP karena hanya butuh operasi numpy sederhana dan tetap efisien pada ~ 29 ribu baris data tanpa solver quadratic-programming eksternal. Hyperparameter: `lambda=1e-3`, `n_epochs=15`, `batch_size=256`.

Validation Strategy

Validasi menggunakan **5-fold cross-validation** (implementasi manual di `data.py`, tanpa `sklearn.model_selection`) dengan metrik **macro F1** (implementasi manual di `metrics.py`, menghitung precision/recall/F1 per kelas lalu dirata-ratakan), agar konsisten dengan metrik leaderboard Kaggle.

Model	CV Macro F1 (mean $\pm$ std)	versi awal
DTL - scratch	0.8747 $\pm$ 0.0082	0.8696
DTL - sklearn	0.8745 $\pm$ 0.0083	0.8703
LR - scratch	0.8466 $\pm$ 0.0090	0.8420
LR - sklearn	0.8467 $\pm$ 0.0091	0.8429
SVM - scratch	0.8456 $\pm$ 0.0087	0.8410
SVM - sklearn	0.8470 $\pm$ 0.0092	0.8427

Ketiga implementasi from-scratch berada sangat dekat dengan versi scikit-learn-nya (selisih ≤ 0.002), memvalidasi kebenaran implementasi - perbedaan kecil yang tersisa wajar karena scikit-learn memakai detail numerik/heuristik internal yang sedikit berbeda (mis. tie-breaking pada split yang identik gain-nya, solver LR berbasis LBFGS vs gradient descent polos di sini). Model terbaik pada CV adalah **DTL-scratch (0.8747)**, dipakai untuk generate `result/submission.csv` (distribusi prediksi test: 5.782 ditolak / 1.418 disetujui dari 7.200 baris).

Skor leaderboard Kaggle: 0.85341 dari submission versi awal (public leaderboard, dihitung dari $\pm 30\%$ data test = ± 2.160 baris; skor final dihitung dari 70% sisanya setelah kompetisi ditutup).

Berapa besar sebenarnya noise pada public leaderboard?

Selisih CV lokal (0.8696) terhadap public LB (0.8534) awalnya dibaca sebagai *generalization gap*. Untuk mengeceknya secara kuantitatif, dilakukan simulasi: satu model yang **sama persis** diskor pada 2.000 subset acak berukuran 2.160 baris dari prediksi *out-of-fold* - meniru "undian" 30% data yang menentukan public leaderboard. Seluruh sebaran di bawah murni keberuntungan sampling, bukan perbedaan kualitas model:

Ukuran	Nilai
Skor pada seluruh 28.800 baris out-of-fold	0.8747
Rata-rata pada sampel 2.160 baris	0.8745
Standar deviasi	0.0085
Rentang 90%	0.8606 - 0.8885
P(sampel terlihat ≥ 0.89)	3,2%

Std 0.0085 berarti **selisih ± 0.02 antar peserta bisa terjadi tanpa perbedaan model sama sekali**, sehingga skor 0.8534 bukan tanda bug melainkan undian yang kurang beruntung. Namun noise saja tidak cukup menjelaskan skor 0.89 - lihat analisis plafon di bawah.

Di mana sebenarnya plafon dataset ini?

Plafon diukur lewat scikit-learn sebagai *oracle* pembanding cepat (submission tetap memakai model from-scratch):

Pendekatan	CV Macro F1
Grid penuh <code>max_depth</code> $\times$ <code>min_samples_leaf</code> (30 kombinasi)	0.8703
CART + cost-complexity pruning (dipakai)	0.8747
Tree khusus untuk subproblem sulit saja	0.8745
Kriteria entropy / log-loss (ganti gini)	0.8704
SVM kernel RBF (Nystroem/RFF & SVC eksak)	0.8642-0.8672
Gradient boosting dituning - <i>terlarang, diagnostik saja</i>	0.8855

Enam pendekatan berbeda tidak ada yang menembus 0.8747, termasuk dua yang paling menjanjikan: SVM non-linear justru **lebih buruk** (kapasitas model bukan faktor pembatas), dan melatih tree khusus hanya pada 14.171 baris sulit memberi 0.8745 - sama saja, karena tree global memang sudah melakukan split pemisah itu sendiri. Dalam batasan aturan, **plafonnya benar-benar di ~ 0.873 - 0.875** .

Namun datanya sendiri belum habis: gradient boosting yang dituning mencapai **0.8855**. Selisih ± 0.012 itu nyata, tetapi hanya dapat diambil lewat *ensemble aditif* yang **dilarang** pada task ini. (Catatan koreksi: ensemble dengan hyperparameter default hanya mencapai 0.877, dan sempat membuat plafonnya terlihat jauh lebih rendah daripada yang sebenarnya - pelajaran bahwa mengukur plafon dengan baseline yang belum dituning bisa menyesatkan.) Konsekuensinya untuk membaca leaderboard: model 0.874 hanya 3,2% berpeluang terlihat ≥ 0.89 , sementara model boosting 0.8855 berpeluang $\pm 30\%$. Skor 0.89 karena itu lebih mungkin berasal dari model di luar aturan daripada dari keberuntungan sampling semata.

Percobaan yang Gagal

Class-balanced sample weighting ($w_c = n / (n_{\text{classes}} \cdot n_c)$) pada ketiga algoritma - hipotesis awal: karena metrik evaluasi adalah macro F1 pada data imbalanced ($\pm 3.5:1$), memberi bobot lebih besar ke kelas minoritas (disetujui) saat training seharusnya membantu. Hasil aktualnya justru **sebaliknya**, macro F1 CV turun cukup jauh di ketiga model:

Model	Dengan weighting	Tanpa weighting
DTL - scratch	0.8452	0.8747
LR - scratch	0.8207	0.8466
SVM - scratch	0.8208	0.8456

Penyebabnya jelas setelah melihat pembelahan data pada Problem Overview: **ketimpangan 3,5:1 itu palsu**, seluruhnya berasal dari cabang deterministik yang diselesaikan satu split, sementara subproblem yang benar-benar dipelajari sudah **nyaris seimbang 55:45**. Class weighting menaikkan bobot kelas minoritas secara *global*, padahal kelas minoritas hanya hidup di subproblem yang tidak butuh koreksi apa pun - jadi yang terjadi adalah menggeser batas keputusan pada bagian data yang sudah seimbang dan mengorbankan prediksi yang sebelumnya benar. Pelajarannya: periksa dulu **di mana** ketimpangan berada; rasio kelas global menyesatkan bila sebagian data sudah deterministik.

Decision-threshold tuning - hipotesis: cutoff 0.5 bukan optimum untuk macro F1. Threshold dicari pada 15% data yang ditahan dari bagian training (fold yang diskor tidak ikut menentukan threshold-nya sendiri, jadi bebas leakage). Hasil **0.8739 $\rightarrow$ 0.8736**, tidak membaik. Yang informatif justru threshold terpilihnya: `[0.31, 0.28, 0.45, 0.45, 0.43]` - berayun jauh antar fold, tanda proses tuning sedang mengejar noise sampling-nya sendiri, bukan optimum nyata.

Feature engineering tambahan - empat fitur turunan ($\text{defaults} \times \text{loan_percent_income}$, $\text{credit_score} \times \text{cred_hist_length}$, $\text{emp_exp} / \text{age}$, $\text{loan_percent_income}^2$): **0.8747** → **0.8739**, di dalam noise antar-fold (std

± 0.008). Masuk akal - CART sudah bisa split pada kolom mentahnya, jadi hasil kali buatan tangan tidak menambah informasi baru. Kontras dengan **log1p** yang berhasil justru karena mengatasi masalah *skala*, bukan menambah sinyal.

SVM kernel non-linear (RBF) - hipotesis: batas keputusan non-linear mengalahkan tree yang hanya memotong sejajar sumbu. Diuji via random Fourier features & SVC RBF eksak (sweep **gamma** 0.05-0.5): terbaik **0.8672**, di bawah single tree. Kapasitas model bukan faktor pembatas di sini.

Pengembangan Lebih Lanjut

Diagnostik menunjukkan sinyal yang tersisa (± 0.012 menuju 0.8855) hanya terjangkau oleh ensemble aditif yang dilarang, sehingga arah pengembangan di dalam aturan bukan lagi menaikkan kapasitas model:

- **Split multivariat (oblique)**. Satu-satunya keunggulan struktural boosting yang belum ditiru adalah kemampuannya menyusun batas keputusan miring lewat banyak split aditif. Sebuah *single tree* dengan split berbasis kombinasi linear fitur (mis. OC1) tetap sebuah decision tree, jadi masih memenuhi batasan, dan merupakan kandidat paling menjanjikan yang belum diuji.
- **Pemilihan **alpha** secara nested**. Saat ini **ccp_alpha** dipilih pada CV yang sama yang dilaporkan sehingga skornya sedikit optimistis; nested CV akan memberi estimasi tak bias.
- **LR**: optimizer adaptif (Adam/RMSProp) untuk konvergensi lebih cepat, regularisasi L1 untuk feature selection otomatis.
- **Menerima batas statistik**. Dengan std public LB ± 0.0085 , mengejar selisih < 0.01 di leaderboard adalah mengejar noise. Usaha lebih berharga dialokasikan ke kualitas analisis - yang juga dinilai - daripada ke optimasi yang tidak dapat dibedakan dari keberuntungan undian.

Referensi

- [1] GeeksforGeeks, "CART (Classification And Regression Tree) in Machine Learning," GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/cart-classification-and-regression-tree-in-machine-learning/> [Accessed: Aug. 7, 2026].
- [2] GeeksforGeeks, "Understanding Logistic Regression," GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/> [Accessed: Aug. 7, 2026].
- [3] GeeksforGeeks, "Support Vector Machine (SVM) Algorithm," GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/> [Accessed: Aug. 7, 2026].
- [4] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth International Group, 1984, ch. 3 ("Right Sized Trees and Honest Estimates") - sumber asli minimal cost-complexity / weakest-link pruning.
- [5] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York: Springer, 2009, §9.2.2 ("Tree-Based Methods - Pruning"), pp. 307-308. Available: <https://hastie.su.domains/ElemStatLearn/>
- [6] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: Primal Estimated sub-GrAdient SOLver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3-30, Mar. 2011. DOI: 10.1007/s10107-010-0420-4. Available: <https://link.springer.com/article/10.1007/s10107-010-0420-4>